

Shor's 1994 Quantum Factoring Algorithm: History, Mathematics, and Simulations

Amey Bhatuse
Sergio Castañeiras Morales
David González Lociga
José Ernesto Ramos Sánchez

Advanced Quantum Mechanics
Master in Quantum Science and Technology Barcelona

October 21, 2025

Quantum Advantage

1986
R. Feynman

May, 1994
D. Simon

November, 1994
P. Shor

Algorithms for Quantum Computation: Discrete Logarithms and Factoring

Peter W. Shor
AT&T Bell Labs
Room 2D-149
600 Mountain Ave.
Murray Hill, NJ 07974, USA

Abstract

A computer is generally considered to be a universal computational device; i.e., it is believed able to simulate any physical computational device with a cost in computation time of at most a polynomial factor. It is not clear whether this is still true when quantum mechanics is taken into consideration. Several researchers, starting with David Deutsch, have developed models for quantum mechanical computers and have investigated their computational properties. This paper gives Las Vegas algorithms for finding discrete logarithms and factoring integers on a quantum computer that take a number of steps which is polynomial in the input size, e.g., the number of digits of the integer to be factored. These two problems are generally considered hard on a classical computer and have been used as the basis of several proposed cryptosystems. (We thus give the first examples of quantum cryptanalysis.)

1 Introduction

Since the discovery of quantum mechanics, people have found the behavior of the laws of probability in quantum mechanics counterintuitive. Because of this behavior, quantum mechanical phenomena behave quite differently than the phenomena of classical physics that we are used to. Feynman seems to have been the first to ask what effect this has on computation [13, 14]. He gave arguments as to why this behavior might make it intrinsically computationally expensive to simulate quantum mechanics on a classical (or von Neumann) computer. He also suggested the possibility of using a computer based on quantum mechanical principles to avoid this problem, thus implicitly asking the converse question: by using quantum mechanics in a computer can you compute more efficiently than on a classical computer. Other early work in the field of quantum mechanics and computing was done by Benioff

[1, 2]. Although he did not ask whether quantum mechanics conferred extra power to computation, he did show that a Turing machine could be simulated by the reversible unitary evolution of a quantum process, which is a necessary prerequisite for quantum computation. Deutsch [9, 10] was the first to give an explicit model of quantum computation. He defined both quantum Turing machines and quantum circuits and investigated some of their properties.

The next part of this paper discusses how quantum computation relates to classical complexity classes. We will thus first give a brief intuitive discussion of complexity classes for those readers who do not have this background. There are generally two resources which limit the ability of computers to solve large problems: time and space (i.e., memory). The field of analysis of algorithms considers the asymptotic demands that algorithms make for these resources as a function of the problem size. Theoretical computer scientists generally classify algorithms as efficient when the number of steps of the algorithm grows as a polynomial in the size of the input. The class of problems which can be solved by efficient algorithms is known as P. This classification has several nice properties. For one thing, it does a reasonable job of reflecting the performance of algorithms in practice (although an algorithm whose running time is the tenth power of the input size, say, is not truly efficient). For another, this classification is nice theoretically, as different reasonable machine models produce the same class P. We will see this behavior resurface in quantum computation, where different models for quantum machines will vary in running times by no more than polynomial factors.

There are also other computational complexity classes discussed in this paper. One of these is PSPACE, which are those problems which can be solved with an amount of memory polynomial in the input size. Another important complexity class is NP, which intuitively is the class of exponential search problems. These are problems which may require the search of an exponential size space to find

Mathematical formulation

Definition

Given $n \in \mathbb{N}$, the multiplicative group of integers modulo n is defined as

$$(\mathbb{Z}/n\mathbb{Z})^* = \{[x] \in \mathbb{Z}/n\mathbb{Z} : \gcd(x, n) = 1\}.$$

For example:

$$(\mathbb{Z}/12\mathbb{Z})^* = \{[1], [5], [7], [11]\}.$$

Factorization

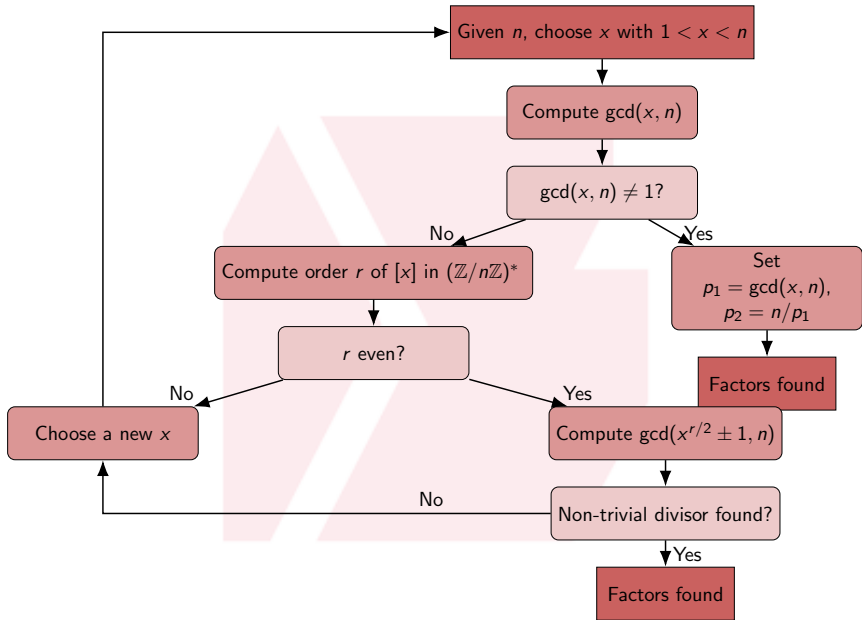
Choose $1 < x < n$
a candidate divisor of n
such that $\gcd(x, n) = 1$.

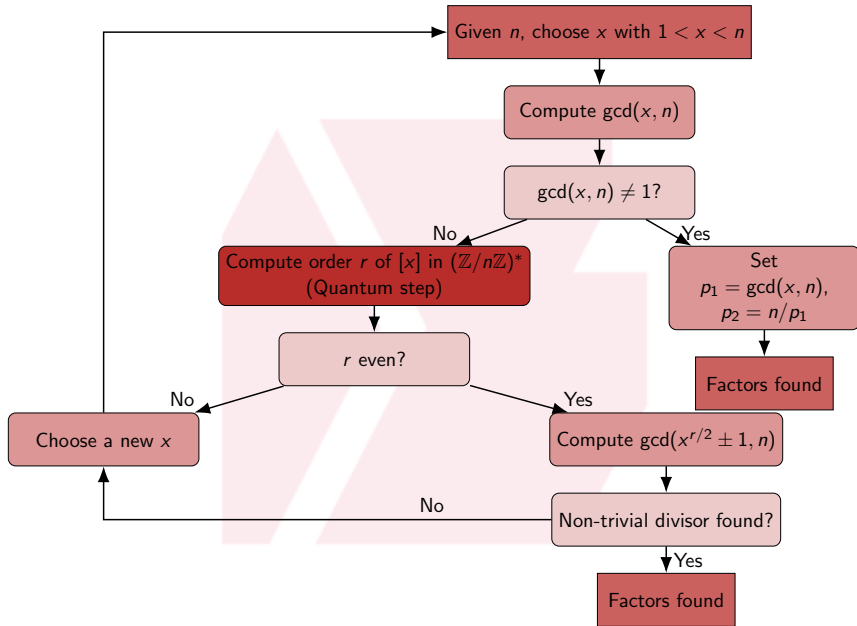
Order-finding
(Quantum subroutine)

Goal: given $n \in \mathbb{N}$ find $p_1, p_2 \in \mathbb{N}$
such that $n = p_1 \cdot p_2$.

Goal: given $n \in \mathbb{N}$ and $[x] \in (\mathbb{Z}/n\mathbb{Z})^*$,
find the smallest $r \in \mathbb{N}$
such that $x^r \equiv 1 \pmod{n}$.

Once r is obtained, a classical algorithm finds a divisor of n !





Implementation of Shor's algorithm

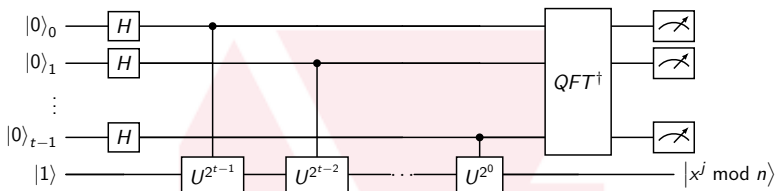


Figure 1: Quantum circuit implementing Shor's algorithm.

Quantum Fourier Transform

Let $\{|k\rangle\}_{k=0}^{N-1}$ be an orthonormal basis. We define the *Quantum Fourier Transform* (QFT) as the unitary operation U_{QFT} such that its action on the basis is,

$$U_{QFT} |r\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{i\frac{2\pi}{N}kr} |k\rangle.$$

Implementation of Shor's algorithm

- State before $QFT^\dagger$: $|\phi\rangle = \frac{1}{\sqrt{r2^t}} \sum_{j=0}^{2^t-1} |j\rangle \sum_{d=0}^{r-1} e^{i\frac{2\pi jd}{r}} |u_d\rangle = \frac{1}{\sqrt{2^t}} \sum_{j=0}^{2^t-1} |j\rangle |x^j \bmod n\rangle$, where $|u_d\rangle$ are the eigenvectors of the unitary operator $U|y\rangle = |xy \bmod n\rangle$.
- Final state: $|\psi\rangle = \frac{1}{\sqrt{r}} \sum_{d=0}^{r-1} |\tilde{c}/q\rangle |u_d\rangle$, where $q = 2^t$ and $\tilde{c}/q$ is an estimation of d/r precise to t bits. We will determine d/r from the estimation $\tilde{c}/q$ using the continued fractions expansion.

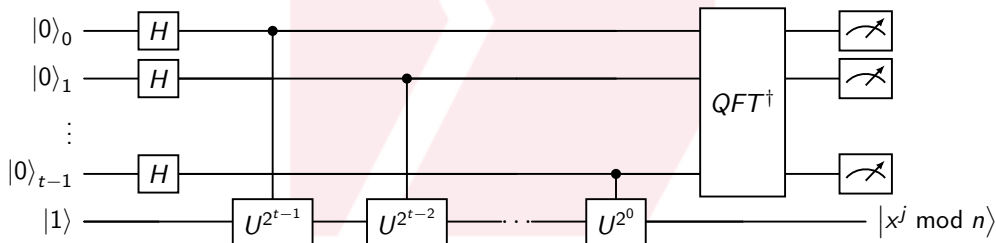


Figure 2: Quantum circuit implementing Shor's algorithm.

Implementation of Shor's algorithm

Probability distribution

The probability of measuring $\tilde{c}/q$ is calculated in the Quantum Phase Estimation (QPE) algorithm and it is given by,

$$\mathbb{P}(\tilde{c}) = \frac{1}{r2^{2t}} \sum_{d=0}^{r-1} \frac{\sin^2 \left(\pi \left(\frac{d}{r} 2^t - \tilde{c} \right) \right)}{\sin^2 \left(\pi \left(\frac{d}{r} - \frac{\tilde{c}}{2^t} \right) \right)}.$$

Example. Factoring $n=55$

- We will begin by choosing $x = 2$, which is coprime to $n = 55$.

How many qubits should we use?

To ensure that we can find d/r via the continued fractions expansion, we need at least,

$$t = 2L + 1,$$

with $L = \lceil \log_2(n) \rceil$.

- In our example, $L = 6$ and we need at least $t = 13$ qubits.

Example. Factoring $n = 55$

[Qiskit Code](#)

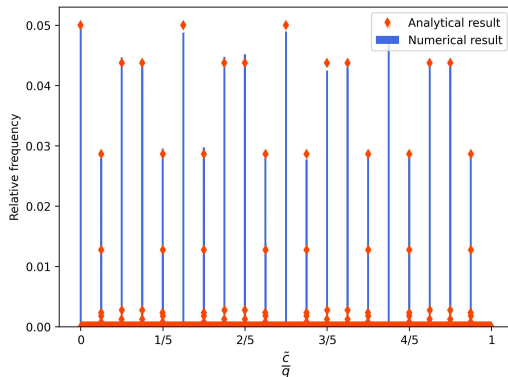


Figure 3: Comparison of results from running Shor's algorithm on $N = 55$ for $t=13$ and 2^{14} iterations with the plot for the analytical formula for estimation precision

Example. Factoring $n = 55$

In Table 1 we illustrate the classical treatment for some of the most probable results $\tilde{c}/q$ of the quantum subroutine:

$\tilde{c}/q$	d/r	r	Relative frequency
0.04993	1/20	20	0.029
0.09998	1/10	10	0.045
0.80005	4/5	5	0.029
0.25000	1/4	4	0.049

Table 1: Simulation results of $\tilde{c}/q$ with their corresponding relative frequency, the resulting d/r from continued fractions expansion and the estimation for r .

- One source of failure is that d and r might share common factors that simplify. This occurs precisely when the results $r = 10, 5, 4$ are obtained.

Example. Factoring $n = 55$

Since the correct order $r = 20$ is even, we can continue with the classical part of the algorithm.

Classical post-processing in Shor's algorithm

$$r = 20 \Rightarrow \begin{cases} \gcd(2^{20/2} - 1, 55) = 11 \\ \gcd(2^{20/2} + 1, 55) = 5 \end{cases} \Rightarrow 55 = 5 \times 11$$

Conclusions

- We have reviewed Shor's original paper and detailed the mathematical framework underneath the algorithm.
- We have reproduced Shor's analytical probability distribution formulation of measuring $\frac{\tilde{c}}{q}$.
- We have verified the analytical expression with numerical results from a practical implementation of the algorithm using the Qiskit framework.



Thank you for your attention!

Backup slides

QFT circuit implementation

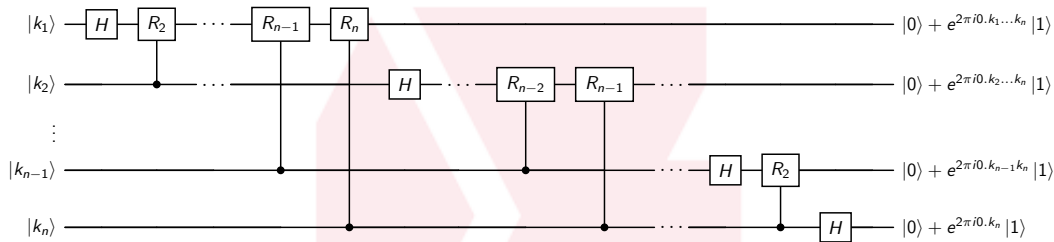


Figure 4: Quantum circuit implementing the Quantum Fourier Transform (QFT). The output states are shown unnormalized (the $1/\sqrt{2}$ factors are omitted). Controlled phase shift gates CR_k are represented by phase shift gates R_k acting on the target qubit, with filled dots indicating their respective control qubits. Notice how by reading the output qubits downwards, we obtain the QFT in the reversed order, so a final SWAP gate is required in order to obtain the desired result.

How does the precision scale with t ?

If we use t qubits in the first register,

$$t = 2L + 1 + \left\lceil \log_2 \left(2 + \frac{1}{2\epsilon} \right) \right\rceil,$$

we will obtain an estimate $\tilde{c}/q$ of the phase d/r accurate to $2L + 1$ bits with probability at least $(1 - \epsilon)/r$.